



COMPUTER
ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	SECTION:	CPE21S1
DATE PERFORMED:	August 4, 2026	DATE SUBMITTED:	August 4, 2026
NAME:	Benj Federic C. De Leon	INSTRUCTOR:	Engr. Jimlord M. Quejado

ACTIVITY NO. 5.1 **[ACTIVITY NAME]**

1. PROCEDURE OUTPUT

ILO A.

INPUT:

```
#include <iostream>
#include <queue>

//prototype
void display(std::queue<char> r);

int main() {

    std::queue<char> q;

    //push
    q.push('c'); // front
    q.push('p');
    q.push('e');
    q.push('0');
    q.push('1');
    q.push('0'); // rear

    //empty()
    std::cout<<"Is the queue empty: " << q.empty()<<std::endl;

    //size
    std::cout<<"Size of the queue: " << q.size()<<std::endl;
```

```
//front
std::cout<< "Front of the queue: "<<q.front()<<std::endl;
// front

//back
std::cout<<"Back of the queue: "<<q.back()<<std::endl;

//pop
std::cout<<"Pop: ";
q.pop();

//push another element
std::cout<< "Push another element: ";
q.push('b');

//display
std::cout<<"Displaying element/s: ";
display(q);

//delete the queue
while(!q.empty()){
    q.pop();
}

}

void display(std::queue<char> r){
    // create a copy of the queue
    std::queue<char> c = r;

    while(!c.empty()){
        std::cout << " "<<c.front();
        c.pop();
    }
    std::cout<<"\n";
```

	<pre> }</pre>
OUTPUT:	<pre> > cd "c:\Use Is the queue empty: 0 Size of the queue: 6 Front of the queue: c Back of the queue: 0 Pop: Push another element: Displaying element/s: p e 0 1 0 b</pre>

Code analysis:

This file tests C++'s built-in std:: queue. It pushes 6 characters in, then runs through every basic queue operation, it checks if it's **empty**, checking **size**, peeking at **front/back**, **popping** one off, **pushing** one more, then **displaying** everything.

ILO B.

INPUT:		
	main.cpp <pre> #include <iostream> #include "queue1.h" int main() { qNode<char>* front = nullptr; qNode<char>* back = nullptr; enqueue(&front, &back, 'J'); std::cout<< front->data << "</pre>	queue1.h <pre> #ifndef QUEUE_H #define QUEUE_H #include <iostream> template <typename T> class qNode{ public: T data; qNode *next; }; //creating a new node. template <typename T></pre>

```
"<<back->data<<std::endl;
    enqueue(&front, &back,
'I');
    std::cout<< front->data
<< "
"<<back->data<<std::endl;
    enqueue(&front, &back,
'M');
    std::cout<< front->data
<< "
"<<back->data<<std::endl;
    dequeue(&front, &back);
    std::cout<<front->data<<
" "<< back->data<<std::endl;

return 0;
}
```

```
qNode<T>* new_node(T
newdata){
    qNode<T>* newNode = new
qNode<T>;
    newNode->data = newdata;
    newNode->next = nullptr;
    return newNode;
}

template<typename T>
void enqueue (qNode<T>*&
frontPtr, qNode<T>*& backPtr,
T newData){
    //creating a NEW NODE
    qNode<T>* newNode =
new_node(newData);

    //INSERTING TO AN EMPTY
QUEUE
    if((*frontPtr) == nullptr
&& (*backPtr)==nullptr){
        (*frontPtr) =
newNode;
        (*backPtr) = newNode;
    }
    //inserting an item into
a non empty queue
    else {
        // point the backPtr
NEXT TO THE newNode
        (*backPtr)->next =
newNode;
        (*backPtr) = newNode;
    }
}
```

```
template<typename T>
void dequeue (qNode<T>**
frontPtr, qNode<T>**
backPtr){
    //check if the queue is
empty to prevent crashes
    if((*frontPtr) ==
nullptr){
        return;
    }

    //createa temporary node
to store the node to be
deleted
    qNode<T>* deleteNode =
nullptr;
    deleteNode = (*frontPtr);

    //check if the queue is
only 1 node
    if((*frontPtr) ==
(*backPtr)){
        (*frontPtr) =
nullptr;
        (*backPtr) = nullptr;
        delete deleteNode;
        return;
    }

    //deleting of the node
    (*frontPtr) =
deleteNode->next;
    delete deleteNode;
}
```

```
//display all elements in
the list
template<typename T>
void display (qNode<T>*
frontPtr){
    qNode<T> *temp =
frontPtr;

    while(temp !=nullptr){
        std::cout<< " " <<
temp->data;
        temp = temp->next;
    }
}

//return the front
template<typename T>
T front(qNode<T>* frontPtr){
    return frontPtr->data;
}

#endif
```

OUTPUT:

```
out=Microsoft-MIEngine
J J
J I
J M
I M
PS C:\Users\TIP06\Docu
```

Code analysis:



This file calls your enqueue three times and dequeue once, printing front/back after each call.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.